

1. PROJECT SYNOPSIS

In today's digital-driven business environment, technology plays a vital role in streamlining day-to-day operations and improving customer service. The Bakery Management System is a modern software solution designed to automate and simplify bakery operations efficiently.

Developed using Python and integrated with a MySQL database, this system helps manage orders, inventory, customer interactions, and payment processing in a seamless manner.

Implemented for the premium bakery “**Dounce Lune**”, the system replaces traditional manual record-keeping methods, reducing errors, saving time, and ensuring smooth operations even during peak business hours.

By providing an interactive and user-friendly interface, the Bakery Management System enhances both administrative control and customer experience. Customers can easily browse products, place orders, track previous purchases, cancel orders if needed, and provide feedback, while administrators can efficiently manage inventory, update product details, monitor orders, and maintain accurate business records. With added security features and flexible payment options, the system ensures safe and convenient transactions, making it a reliable solution for modern bakery management.

Advantages of the Bakery Management System:

- **Automated Order Management**

Streamlines the entire order process from placement to confirmation, reducing manual effort and processing time.

- **Efficient Inventory Control**

Allows administrators to monitor stock levels, update product availability, and prevent overstocking or shortages.

- **User-Friendly Customer Interface**

Enables customers to easily view products, place orders, check order history, and submit ratings or feedback.

- **Secure Authentication & Data Protection**

Incorporates user authentication and CAPTCHA verification to ensure authorized access and protect sensitive data.

- **Multiple Payment Options**

Supports flexible payment methods such as credit/debit cards, UPI, and cash on delivery for customer convenience.

- **Reduced Errors & Improved Accuracy**

Patients can store prescriptions, reports, and consultation history for future use, reducing paperwork.

- **Reduced Human Error & Better Management**

Minimizes human errors associated with handwritten records and manual calculations.

- **Time-Saving & Operational Efficiency**

Allows easy integration of additional features such as home delivery, discount management, and sales reporting.

- **Enhanced Customer Satisfaction**

Provides a smooth ordering and payment experience, leading to better customer engagement and retention.

2. SOFTWARE & HARDWARE REQUIREMENTS

I. HARDWARE SPECIFICATIONS

- ⇒ **Processor** : Intel Core i3 or higher to support smooth execution of application processes
- ⇒ **RAM** : Minimum 4GB to handle application and database operations
- ⇒ **Storage** : Minimum 100GB hard disk space for storing application files and database records
- ⇒ **Input Devices** : Standard keyboard and mouse for data entry and system navigation
- ⇒ **Output Devices** : Monitor with a minimum display resolution of 1024×768 for proper interface visibility

II. SOFTWARE SPECIFICATIONS

- ⇒ **Operating system** : Windows 10 or later versions / Linux distributions compatible with Python and MySQL
- ⇒ **Programming Language** : Python 3.13.5, used for developing the core application logic due to its simplicity and flexibility
- ⇒ **Database Management** : MySQL Server for secure, reliable, and efficient storage of customer, product, order, and transaction data
- ⇒ **Database Connectivity** : MySQL Connector for Python to enable seamless interaction between the Python application and the MySQL database

3. WORKING OF THE PROJECT

1. MAIN PROGRAM INITIALISATION

- The program begins execution inside a **main while loop** with the condition set to True.
- This loop ensures that the application continues to run until the user explicitly chooses to exit.
- The program is function-based and primarily relies on three user-defined functions:
 - login ()
 - customer ()
 - admin ()

2. LOGIN FUNCTION

- The login() function is invoked at the start of the program.
- It identifies whether the user is a **Customer** or an **Administrator**.
- The function returns two values:
 - User identity (Customer / Administrator)
 - A flag value is set to False or True indicating login success or failure respectively.

a) Customer Login / Sign-Up

- Provides the customer with two options: **Login** or **Sign-Up**

Customer Login:

- Collects the customer username and verifies its existence in the database
- If the username exists, the password is requested and validated
- A CAPTCHA verification is performed to enhance security

- Successful CAPTCHA validation logs the customer into the system
- If credentials are invalid, the user is prompted to retry or sign up

Customer Sign-Up:

- Automatically assigns a unique customer ID
- Collects customer details such as name, gender, and address
- Allows the customer to create a username and password
- Stores the collected data securely in the customer database table

b) Administrator Login

- Collects the admin ID and password
- Validates credentials against existing database records
- Performs CAPTCHA verification for added security
- Successful validation grants administrator access

3. CUSTOMER MODULE

- Activated when the logged-in user is identified as a **Customer**
- Displays a menu inside a loop, allowing the customer to perform multiple operations

(i) Place an Order

- Collects and verifies customer name and mobile number
- If customer details are not found, an option to create an account is provided
- Displays the list of available products with price and quantity
- Allows ordering of multiple products using a loop
- Generates a unique order number and stores order details in the database
- Updates product stock after order confirmation

- Calculates the total bill amount
- Invokes the payment() function

Payment Process:

- Offers payment modes: Card, UPI, or Cash on Delivery
- Collects required payment details
- Displays confirmation with transaction date and time

(ii) View Past Orders

- Verifies customer details using name and mobile number
- Retrieves customer ID from the database
- Displays previous order details if records exist
- Shows an appropriate message if no orders are found

(iii) Cancel an Order

- Verifies customer details
- Requests and validates the order number
- Deletes the order record if valid
- Displays confirmation or error message accordingly

(iv) Rate an Order

- Verifies customer and order details
- Accepts rating input between 1 and 5
- Validates input and updates the rating field in the database

(v) Logout

- Exits the customer menu loop
- Returns control to the main program

4. ADMINISTRATOR MODULE

- Activated when the logged-in user is identified as an **Administrator**
- Provides administrative operations through a looped menu

(i) Add New Product

- Collects product details such as ID, name, price, and quantity
- Validates data types
- Inserts product information into the database

(ii) Update Product Details

- Requests product ID and validates its existence
- Allows updating product name, price, or quantity
- Updates the database using SQL queries
- Displays confirmation upon successful update

(iii) Delete Product

- Collects and validates product ID
- Removes the corresponding product record from the database

(iv) View all Customer Orders

- Retrieves and displays all orders placed by customers
- Displays a message if no orders are available

(v) View Orders of a Specific Customer

- Verifies customer details
- Retrieves customer ID
- Displays all orders associated with that customer

(vi) Logout

- Exits the administrator menu loop
- Returns control to the main program

5. PROGRAM TERMINATION

- After logout, the user is given an option to **exit the application**
- If exit is selected, the main while loop is terminated
- The program ends execution gracefully

4. SOURCE CODE

```
# ===== Final Douce Lune Bakery System =====

import random

import string

from datetime import date, datetime

import mysql.connector as mc


# Global flag

flag = True


# ----- CAPTCHA -----

def simple_captcha():

    code = ".join(random.choices(string.ascii_letters + string.digits, k=8))

    print(f"\n🔒 Simple Verification — Enter this code exactly:\n[{code}]" )

    user_input = input("Enter the code: ").strip()

    if user_input == code:

        print("✅ Verification passed!")

        return True

    else:

        print("\n❌ CAPTCHA failed. Try again.\n")

        return False
```

```

# ----- LOGIN -----

def login():

    global flag

    flag = True

    global cid

    cid = None

    while flag:

        identifier = input("Are you a customer or administrator? ")

        if identifier.lower() == 'customer':

            entry = input("\nDo you want to Login or Sign-Up? ")

            # Customer - Login

            if entry.lower() == 'login':

                while flag:

                    global username

                    username = input("Enter your Username: ")

                    cur.execute("SELECT USERNAME FROM CUSTOMER WHERE
USERNAME=%s", (username.lower(),))

                    result = cur.fetchone()

                    if result:

                        passwd = input("Enter your password: ")

                        cur.execute("SELECT PASSWORD FROM CUSTOMER WHERE
USERNAME=%s", (username,))

                        result1 = cur.fetchone()

                        if result1 and passwd == result1[0]:

                            if simple_captcha():

```

```

        print("\n✔ Logged In Successfully!\n")

        cur.execute("SELECT CID FROM CUSTOMER WHERE
USERNAME=%s", (username,))

        cid = cur.fetchone()[0]

        flag = False

    else:

        print("\n✗ Invalid Password!\n")

    else:

        print(f"\n✗ Username {username} not found!\n")

        ans = input("Try Re-Login (#) or Sign-Up (@)? ")

        if ans == "#":

            continue

        elif ans == "@":

            cur.execute("SELECT CID FROM CUSTOMER;")

            data = cur.fetchall()

            cust_id = data[-1][0]+1

            name = input("\nEnter your name: ")

            gen = input('Enter your gender: ')

            while True:

                cont = input('Enter your contact number: ')

                if not(len(cont)>=10 and len(cont)<=13):

                    print('\nEnter a valid contact\n')

                else:

                    break

```

```

        add = input('Enter your address: ')

        c_code = int(input('Enter your city code: '))

        while True:

            username = input('Enter a username (Gmail Id): ')

            if '@' in username:

                break

            else:

                print('\nEnter a valid username!\n')

            passwd = input("Create password: ")

            cur.execute(f'INSERT INTO CUSTOMER VALUES
({cust_id},{name},{gen}', '{cont}', '{add}', {c_code}, '{username}',
'{passwd}');")

            db.commit()

            print("\n✔ Sign-Up successful!\n")

        else:

            break

# Customer - Sign-Up

elif entry.lower() == 'sign-up':

    cur.execute("SELECT CID FROM CUSTOMER;")

    data = cur.fetchall()

    cust_id = data[-1][0]+1

    name = input("\nEnter your name: ")

    gen = input('Enter your gender: ')

    while True:

        cont = input('Enter your contact number: ')

```

```

        if not(len(cont)>=10 and len(cont)<=13):

            print("\nEnter a valid contact\n")

        else:

            break

    add = input('Enter your address: ')

    c_code = int(input('Enter your city code: '))

    while True:

        username = input('Enter a username (Gmail Id): ')

        if '@' in username:

            break

        else:

            print("\nEnter a valid username!\n")

    passwd = input("Create password: ")

    cur.execute(f"INSERT INTO CUSTOMER VALUES ({cust_id},{name},{gen},
'{cont}', '{add}', {c_code}, '{username}', '{passwd}');")

    db.commit()

    print("\n✔ Sign-Up successful!\n")

else:

    print("\nEnter a valid choice\n")

# Admin - Login

elif identifier.lower() == 'administrator':

    try:

        aid = int(input("\nEnter Admin ID: "))

    except ValueError:

        print("Invalid Admin ID.")

```

```

        continue

    passwd = input("Enter Password: ")

    cur.execute("SELECT PASSWORD FROM ADMIN WHERE AdNo=%s", (aid,))

    result = cur.fetchone()

    if result and passwd == result[0]:

        if simple_captcha():

            print("\n✓ Admin Logged In Successfully!\n")

            flag = False

        else:

            print("\n✗ Invalid Admin credentials.\n")

        else:

            print("\nEnter either 'customer' or 'administrator'.\n")

    return identifier, flag


# ----- ADMIN -----

def Admin():

    while True:

        print("===== ADMIN MENU =====")

        print("1. Add New Product\n2. Update Product Details\n3. Delete Product\n4. View All  
Customer Orders\n5. View Orders of a Customer\n6. Logout")

        choice = input("Choice: ")

        if choice=='1':

            try:

                PdNo=int(input("\nEnter the Product ID: "))

```

```

    PdName=input("Enter the Product Name: ")

    UtPr=float(input("Enter the Unit Price: "))

    Qty=int(input("Enter the Quantity available: "))

except ValueError:

    print("\nInvalid input.\n")

    continue

cur.execute("INSERT INTO PRODUCT (PdNo,PdName,UtPr,QTY) VALUES
(%s,%s,%s,%s)", (PdNo,PdName,UtPr,Qty))

db.commit()

print("\n✔ Product added!\n")

elif choice=='2':

    try:

        PdNo=int(input("\nEnter the Product ID to update: "))

    except ValueError:

        print("\nInvalid Product ID.\n")

        continue

    print("\nChoose the field that needs to be updated\n1. Name\n2. Price\n3. Quantity")

    u=input("Choice: ")

    if u=='1':

        cur.execute("UPDATE PRODUCT SET PdName=%s WHERE
PdNo=%s", (input("New name: "),PdNo))

    elif u=='2':

        try:

            cur.execute("UPDATE PRODUCT SET UtPr=%s WHERE
PdNo=%s", (float(input("New price: ")),PdNo))

```

```

except:

    print("\nInvalid price.\n")

    continue

elif u=='3':

    try:

        cur.execute("UPDATE PRODUCT SET QTY=%s WHERE
PdNo=%s",(int(input("New qty: ")),PdNo))

    except:

        print("\nInvalid quantity.\n")

        continue

else:

    print("\nInvalid option.\n")

    continue

db.commit()

print("\n✔ Updated successfully!\n")

elif choice=='3':

    try:

        PdNo=float(input("\nEnter the Product ID of the product to delete: "))

    except ValueError:

        print("\nInvalid ID.\n")

        continue

    cur.execute("DELETE FROM PRODUCT WHERE PdNo=%s",(PdNo,))

    db.commit()

    print("\n✔ Product deleted!\n")

```

```

elif choice=='4':

    cur.execute("SELECT * FROM ORDERS")

    orders=cur.fetchall()

    if not orders:

        print("\nNo orders.\n")

        continue

    print()

    print('SNO\tOrder Number\tPdNo\tCID\tQTY\tRating\tDate')

    for row in orders:

        for j in range(len(row)):

            if j == 6:

                print(str(row[j]), end='t')

            else:

                print(row[j], end= 't')

        print()

    print()

elif choice=='5':

    cname=input("\nEnter the Customer Name: ")

    contact=input("Enter the Contact No: ")

    cur.execute("SELECT CID FROM CUSTOMER WHERE CNAME=%s AND
CONTACT=%s",(cname,contact))

    res=cur.fetchone()

    if res:

        cid=res[0]

        cur.execute("SELECT * FROM ORDERS WHERE CID=%s",(cid,))

```

```

ords=cur.fetchall()

if ords:

    print('SNO\tOrder Number\tPdNo\tCID\tQTY\tRating\tDate')

    for row in ords:

        for i in range(len(row)):

            if i==6:

                print(row[i], end='\t')

            else:

                print(row[i], end='\t')

        print()

    print()

else:

    print("\nNo orders found.\n")

else:

    print("\nCustomer not found.\n")

elif choice=='6':

    print("\nLogged out.\n")

    break

else:

    print("\nInvalid choice.\n")

```

```

# ----- CUSTOMER -----

def customer():

    while True:

        cur.execute(f'SELECT CNAME FROM CUSTOMER WHERE USERNAME =
'{username}';')

        name = cur.fetchone()[0]

        print(f'HELLO {name}! What would you like to do Today?\n1. Place an Order\n2.
View Past Orders\n3. Cancel an Order\n4. Rate an Order\n5. Logout')

        ch=input("Choice: ")

        if ch=='1':

            cur.execute("SELECT * FROM PRODUCT")

            products=cur.fetchall()

            print("\nMENU")

            [print(f'{p[0]} - {p[1]} ({p[2]}') for p in products]

            cart=[]

            total=0

            while True:

                p=input("\nEnter the Product No: ")

                p=int(p) if p.isdigit() else -1

                if p in [x[0] for x in products]:

                    pd=[x for x in products if x[0]==p][0]

                    qty=int(input("Enter the Quantity you require: "))

                    cart.append([pd[0],pd[1],qty,pd[2]])

                    total+=qty*pd[2]

                else:

```

```

        print("\nInvalid product.\n")

    more=input("\nOrder more? (Y/N): ")

    if more.lower() != 'y':

        break

    cur.execute("SELECT SNO FROM ORDERS ORDER BY SNO DESC LIMIT 1")

    last = cur.fetchone()

    sno = last[0]+1 if last else 1

    t_date=str(date.today())

    num_st = ".join(t_date.split('-'))

    cur.execute(f"SELECT ORD_NUM FROM ORDERS WHERE ORD_NUM LIKE
    '{num_st}%';")

    ord_numbers = cur.fetchall()

    if ord_numbers:

        ord_c = int(ord_numbers[-1][0][-1])

    else:

        ord_c = 1

    for item in cart:

        if ord_c<10:

            ord_num = num_st + '-0'+str(ord_c)

        else:

            ord_num = num_st + '-' +str(ord_c)

        ord_c+=1

        cur.execute("INSERT INTO ORDERS
        (SNO,ORD_NUM,PdNo,CID,QTY,RATING,DATE) VALUES
        (%s,%s,%s,%s,%s,NULL,%s)", (sno,ord_num,item[0],cid,item[2],t_date))

```

```

        cur.execute("UPDATE PRODUCT SET QTY=QTY-%s WHERE PdNo=%s",
(item[2],item[0]))

        sno += 1

        db.commit()

    print(f"\n✔ Order placed! Total = {total}\n")

    payment()

elif ch=='2':

    cur.execute("SELECT PRODUCT.PdName,PRODUCT.UtPr,ORDERS.QTY FROM
ORDERS JOIN PRODUCT ON ORDERS.PdNo=PRODUCT.PdNo WHERE
ORDERS.CID=%s",(cid,))

    past=cur.fetchall()

    if not past:

        print("\nNo past orders.\n")

    else:

        print("\nProduct\t\tPrice\tQty")

        [print(f"{x[0]}\t{x[1]}\t{x[2]}") for x in past]

        print()

elif ch=='3':

    onum=input("\nEnter the Order No: ")

    cur.execute("SELECT * FROM ORDERS WHERE CID=%s AND
ORD_NUM=%s",(cid,onum))

    if not cur.fetchall():

        print("\nInvalid order number.\n")

        continue

    cur.execute("DELETE FROM ORDERS WHERE CID=%s AND
ORD_NUM=%s",(cid,onum))

```

```

        db.commit()

        print("\n✔ Order cancelled!\n")

    elif ch=='4':

        onum=input("\nEnter the Order No: ")

        try:

            rate=int(input("\nRate 1-5: "))

        except:

            print("\nInvalid input.\n")

            continue

        cur.execute("UPDATE ORDERS SET RATING=%s WHERE CID=%s AND
ORD_NUM=%s",(rate,cid,onum))

        db.commit()

        print("\n✔ Thank you for rating!\n")

    elif ch=='5':

        print("\nThank you for choosing Douce Lune Bakery!\n")

        break

    else:

        print("\nInvalid choice.\n")


# ----- PAYMENT -----

def payment():

    print("Choose payment method:\n1. Card\n2. UPI\n3. Cash")

    pay=input("Choice: ")

    t_date=str(date.today())

```

```

time=datetime.now().strftime("%H:%M:%S")

if pay=='1':

    try:

        card=int(input("\nCard No: "))

        cvv=int(input("CVV: "))

    except:

        print("Invalid data.")

        return

    name=input("Name on card: ")

    exp=input("Expiry: ")

    print(f"\n✓ Payment successful on {t_date} at {time}\n")

elif pay=='2':

    while True:

        user=input("\nUPI ID: ")

        if '@ok' in user:

            break

        else:

            print("\nEnter a valid UPI ID!\n")

    try:

        pin=int(input("PIN: "))

    except:

        print("Invalid.")

        return

    print(f"\n✓ Payment successful on {t_date} at {time}\n")

```

```

elif pay=='3':

    print("\n✔ Cash order confirmed, track your order!\n")

else:

    print("\nEnter a valid choice!\n")


# ----- DATABASE CONNECTION -----

db = mc.connect(host='localhost', user='root', password='bunny9953')

cur = db.cursor()

cur.execute("USE BAKERY;")


# ----- MAIN LOOP -----

while True:

    print("Welcome to Douce Lune Bakery - Où chaque bouchée semble être un rêve!")

    iden, f = login()

    if iden.lower()=='customer' and f==False:

        customer()

        ch=input("Exit page? (Y/N): ")

        if ch.lower()=='y':

            break

    elif iden.lower()=='administrator' and f==False:

        Admin()

        ch=input("Exit page? (Y/N): ")

        if ch.lower()=='y':

            break

```

5. OUTPUT

MySQL

```
mysql> USE BAKERY;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_bakery |
+-----+
| admin              |
| customer           |
| orders             |
| product            |
+-----+
4 rows in set (0.030 sec)
```

```
mysql> SELECT * FROM ADMIN;
+-----+-----+-----+-----+-----+-----+
| AdNo | ADM_NAME | JOB          | SALARY | USERNAME                                                                 | PASSWORD |
+-----+-----+-----+-----+-----+-----+
| 101  | Priya Sharma | Manager      | 65000.00 | priya_sharma@admin.doucelunebakery.org | admin1290 |
| 102  | Ravi Menon   | Cashier      | 35000.00 | ravi_menon@admin.doucelunebakery.org  | admin1290 |
| 103  | Anita Rao    | Inventory Supervisor | 42000.00 | anita_rao@admin.doucelunebakery.org   | admin1290 |
| 104  | Vikram Das   | Order Manager | 48000.00 | vikram_das@admin.doucelunebakery.org   | admin1290 |
| 105  | Meghna Iyer  | Head Chef    | 75000.00 | meghna_iyer@admin.doucelunebakery.org  | admin1290 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.014 sec)
```

```
mysql> SELECT * FROM CUSTOMER;
+-----+-----+-----+-----+-----+-----+-----+-----+
| CID | CNAME      | GENDER | CONTACT | ADDRESS          | CITY_CODE | USERNAME                                                                 | PASSWORD |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 201  | Aarav Kumar | M       | 9876543210 | 12, Anna Nagar   | 600126    | aarav_kumar@gmail.com          | aarav2112 |
| 202  | Danny Rajan | F       | 9845012345 | 45, T Nagar      | 600128    | danny.rajan@gmail.com          | dans2231 |
| 203  | Rohan Iyer  | M       | 9789034567 | 7, Velachery Main Road | 600130    | rohaniyer29@gmail.com          | rohan_12. |
| 204  | Diya Sharma | F       | 9360029876 | 22, Adyar Colony | 600131    | diya_sharma2232@gmail.com      | _sharmas |
| 205  | Sanjay Menon | M       | 9003056789 | 18, Besant Nagar | 600134    | sanjay.menon_12@gmail.com      | sandy0012 |
| 206  | Aswatha     | F       | 9874563210 | 13, West Mambalam | 600012    | aswatha_srinivasan12@gmail.com | _aswatha.srinivasan |
+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.009 sec)
```

```
mysql> SELECT * FROM ORDERS;
+-----+-----+-----+-----+-----+-----+-----+
| SNO | ORD_NUM    | PdNo | CID | QTY | RATING | DATE       |
+-----+-----+-----+-----+-----+-----+-----+
| 1   | 20251110-01 | 111  | 201 | 1   | 4.9    | 2025-11-10 |
| 2   | 20251110-02 | 115  | 203 | 1   | 4.0    | 2025-11-10 |
| 3   | 20251110-03 | 120  | 205 | 2   | 5.0    | 2025-11-10 |
| 4   | 20251214-04 | 102  | 206 | 2   | NULL   | 2025-12-14 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.006 sec)
```

```
mysql> SELECT * FROM PRODUCT;
```

PdNo	PdName	UtPr	QTY
101	Butter Croissant	75.50	24
102	Chocolate Croissant	85.00	18
103	Sweet Crepe	120.75	15
104	Macaron Assortment	150.00	30
105	Apple Tart	180.25	10
106	Chocolate Eclair	110.50	18
107	Madeleine Cakes	95.00	22
108	Traditional Baguette	60.00	25
109	Quiche Lorraine	200.00	12
110	French Onion Soup	175.50	8
111	Cheese Platter	250.75	5
112	Chocolate Mousse	140.25	14
113	Lemon Tart	190.00	10
114	French Toast	130.50	16
115	Cafe Latte	90.00	40
116	Espresso	95.00	30
117	Cappuccino	120.50	24
118	Cafe Mocha	135.75	19
119	Cold Brew Coffee	140.00	18
120	Hot Chocolate	110.00	19
121	Iced Lemon Tea	90.00	25
122	Mint Mojito	100.50	20
123	Strawberry Smoothie	150.75	15
124	Mango Frappe	160.00	15
125	Classic Milkshake	120.25	22

```
25 rows in set (0.007 sec)
```

PYTHON

Welcome to Douce Lune Bakery - Où chaque bouchée semble être un rêve!
Are you a customer or administrator? administrator

Enter Admin ID: 102
Enter Password: admin1290

 Simple Verification — Enter this code exactly:
[l1C1BtTm]

Enter the code: l1C1BtTm

☒ Verification passed!

☒ Admin Logged In Successfully!

===== ADMIN MENU =====

1. Add New Product
 2. Update Product Details
 3. Delete Product
 4. View All Customer Orders
 5. View Orders of a Customer
 6. Logout
- Choice: 1

Enter the Product ID: 126
Enter the Product Name: Waffle
Enter the Unit Price: 125.75
Enter the Quantity available: 13

☒ Product added!

===== ADMIN MENU =====

1. Add New Product
 2. Update Product Details
 3. Delete Product
 4. View All Customer Orders
 5. View Orders of a Customer
 6. Logout
- Choice: 2

Enter the Product ID to update: 126

Choose the field that needs to be updated

1. Name
 2. Price
 3. Quantity
- Choice: 1

New name: Chocolate Waffle

☒ Updated successfully!

===== ADMIN MENU =====

1. Add New Product
 2. Update Product Details
 3. Delete Product
 4. View All Customer Orders
 5. View Orders of a Customer
 6. Logout
- Choice: 2

Enter the Product ID to update: 126

Choose the field that needs to be updated

1. Name
2. Price
3. Quantity

Choice: 2

New price: 135

☒ Updated successfully!

===== ADMIN MENU =====

1. Add New Product
2. Update Product Details
3. Delete Product
4. View All Customer Orders
5. View Orders of a Customer
6. Logout

Choice: 2

Enter the Product ID to update: 126

Choose the field that needs to be updated

1. Name
2. Price
3. Quantity

Choice: 3

New qty: 25

☒ Updated successfully!

===== ADMIN MENU =====

1. Add New Product
2. Update Product Details
3. Delete Product
4. View All Customer Orders
5. View Orders of a Customer
6. Logout

Choice: 3

Enter the Product ID of the product to delete: 126

☒ Product deleted!

===== ADMIN MENU =====

1. Add New Product
2. Update Product Details
3. Delete Product
4. View All Customer Orders
5. View Orders of a Customer
6. Logout

Choice: 4

SNO	Order Number	PdNo	CID	QTY	Rating	Date
1	20251110-01	111	201	1	4.9	2025-11-10
2	20251110-02	115	203	1	4.0	2025-11-10
3	20251110-03	120	205	2	5.0	2025-11-10
4	20251214-04	102	206	2	None	2025-12-14

===== ADMIN MENU =====

1. Add New Product
2. Update Product Details
3. Delete Product
4. View All Customer Orders
5. View Orders of a Customer
6. Logout

Choice: 5

Enter the Customer Name: Aswatha

Enter the Contact No: 9874563210

SNO	Order Number	PdNo	CID	QTY	Rating	Date
4	20251214-04	102	206	2	None	2025-12-14

===== ADMIN MENU =====

1. Add New Product
2. Update Product Details
3. Delete Product
4. View All Customer Orders
5. View Orders of a Customer
6. Logout

Choice: 6

Logged out.

Exit page? (Y/N): Y

Welcome to Douce Lune Bakery - Où chaque bouchée semble être un rêve!

Are you a customer or administrator? customer

Do you want to Login or Sign-Up? Sign-Up

Enter your name: Dhanyashree

Enter your gender: F

Enter your contact number: 7893210456

Enter your address: 16, Krishna Nagar

Enter your city code: 600098

Enter a username (Gmail Id): dhanyashree_1245

Enter a valid username!

Enter a username (Gmail Id): dhanyashree_1245@gmail.com

Create password: dhanashreeee

☒ Sign-Up successful!

Are you a customer or administrator? customer

Do you want to Login or Sign-Up? login

Enter your Username: lakshaya_7953@gmail.com

☒ Username lakshaya_7953@gmail.com not found!

Try Re-Login (#) or Sign-Up (@)? #

Enter your Username: dhanyashree_1245@gmail.com

Enter your password: dhanashreeee

 Simple Verification — Enter this code exactly:

[S4Wg4BO0]

Enter the code: S4Wg4BO0

☒ Verification passed!

☒ Logged In Successfully!

HELLO Dhanyashree! What would you like to do Today?

1. Place an Order
2. View Past Orders
3. Cancel an Order
4. Rate an Order
5. Logout

Choice: 1

MENU

- 101 - Butter Croissant (75.5)
- 102 - Chocolate Croissant (85.0)
- 103 - Sweet Crepe (120.75)
- 104 - Macaron Assortment (150.0)
- 105 - Apple Tart (180.25)
- 106 - Chocolate Eclair (110.5)
- 107 - Madeleine Cakes (95.0)
- 108 - Traditional Baguette (60.0)
- 109 - Quiche Lorraine (200.0)
- 110 - French Onion Soup (175.5)
- 111 - Cheese Platter (250.75)
- 112 - Chocolate Mousse (140.25)
- 113 - Lemon Tart (190.0)
- 114 - French Toast (130.5)
- 115 - Cafe Latte (90.0)
- 116 - Espresso (95.0)
- 117 - Cappuccino (120.5)
- 118 - Cafe Mocha (135.75)
- 119 - Cold Brew Coffee (140.0)
- 120 - Hot Chocolate (110.0)
- 121 - Iced Lemon Tea (90.0)
- 122 - Mint Mojito (100.5)
- 123 - Strawberry Smoothie (150.75)
- 124 - Mango Frappe (160.0)
- 125 - Classic Milkshake (120.25)

Enter the Product No: 101

Enter the Quantity you require: 1

Order more? (Y/N): Y

Enter the Product No: 118

Enter the Quantity you require: 1

Order more? (Y/N): Y

Enter the Product No: 120

Enter the Quantity you require: 1

Order more? (Y/N): Y

Enter the Product No: 117

Enter the Quantity you require: 1

Order more? (Y/N): N

☒ Order placed! Total = 441.75

Choose payment method:

1. Card
 2. UPI
 3. Cash
- Choice: 2

UPI ID: 7893210456@okicici

PIN: 106618

☒ Payment successful on 2025-12-26 at 14:57:03

HELLO Dhanyashree! What would you like to do Today?

1. Place an Order
 2. View Past Orders
 3. Cancel an Order
 4. Rate an Order
 5. Logout
- Choice: 2

Product	Price	Qty
Butter Croissant	75.5	1
Cafe Mocha	135.75	1
Hot Chocolate	110.0	1
Cappuccino	120.5	1

HELLO Dhanyashree! What would you like to do Today?

1. Place an Order
 2. View Past Orders
 3. Cancel an Order
 4. Rate an Order
 5. Logout
- Choice: 3

Enter the Order No: 20251226-01

☒ Order cancelled!

HELLO Dhanyashree! What would you like to do Today?

1. Place an Order
 2. View Past Orders
 3. Cancel an Order
 4. Rate an Order
 5. Logout
- Choice: 4

Enter the Order No: 20251226-02

Rate 1-5: 5

☒ Thank you for rating!

HELLO Dhanyashree! What would you like to do Today?

1. Place an Order
2. View Past Orders
3. Cancel an Order
4. Rate an Order
5. Logout

Choice: 5

Thank you for choosing Douce Lune Bakery!

Exit page? (Y/N): Y

6. BIBLIOGRAPHY

1. Google Search Results – Various tutorials and explanations for Python, MySQL, and bakery management systems.
2. Scribd Documents – Notes, sample projects, and guides on Python-based management systems.
3. Python Official Documentation – <https://docs.python.org/3/>
4. MySQL Official Documentation – <https://dev.mysql.com/doc/>
5. TutorialsPoint – Python and MySQL Integration Tutorials
6. YouTube tutorials – Practical demonstrations on Python GUI, database applications, and project development